



TECH CHALLENGE 2

Como transformar milhões de pedidos **em** **um sistema de recomendação de produtos?**



Recomendação de produtos não é um recurso a mais — é motor de receita

26% da receita

vem de compradores que clicam numa recomendação — mesmo sendo só 7% do tráfego

Fonte: Salesforce Shopping Index

10 a 30%

de aumento médio de receita com personalização líder de mercado

Fonte: McKinsey & BCG

91%

dos consumidores compram mais de marcas que oferecem recomendações relevantes

Fonte: pesquisa de mercado em personalização (2025)

É nesse cenário — onde recomendação já não é opcional — que este projeto constrói, do zero, um sistema de recomendação real para o Instacart.

Recomendação genérica, ou aprendida com cada cliente?

Quase sempre genérica. Na maioria das lojas de e-commerce, a recomendação mostra os produtos mais vendidos para todo mundo — sem aprender o padrão de compra de cada cliente.

3,4 milhões

de pedidos analisados, de ~206 mil usuários e ~50 mil produtos

No desafio proposto, processei 32,4 milhões de registros de pedidos do Instacart (dataset público Market Basket Analysis) e construí um modelo que aprende esse padrão por usuário e por produto.

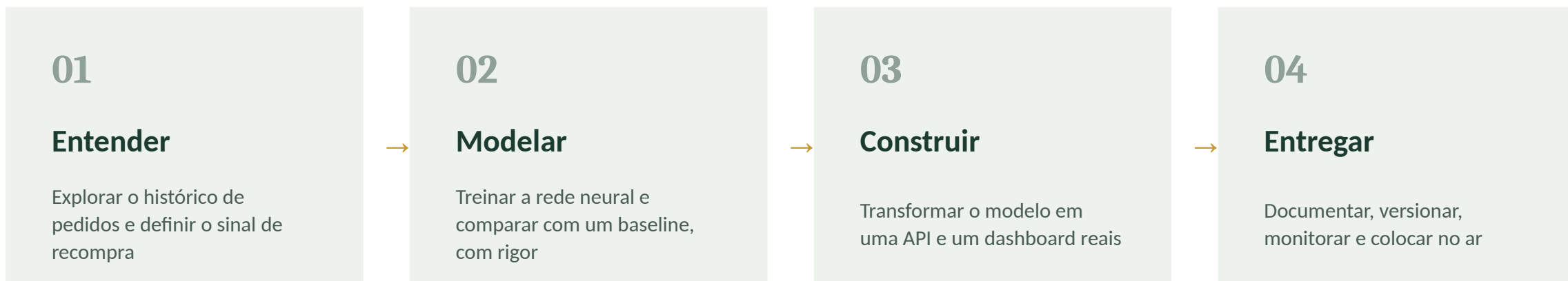
98,8%

dos casos reais de recompra o modelo neural identifica corretamente (recall na validação)

TASK

A missão: não parar no notebook

O desafio não era treinar um modelo qualquer — era construir o ciclo completo de Machine Learning Engineering, da pergunta de negócio a um sistema rodando de verdade.



Dois modelos, resultados bem diferentes

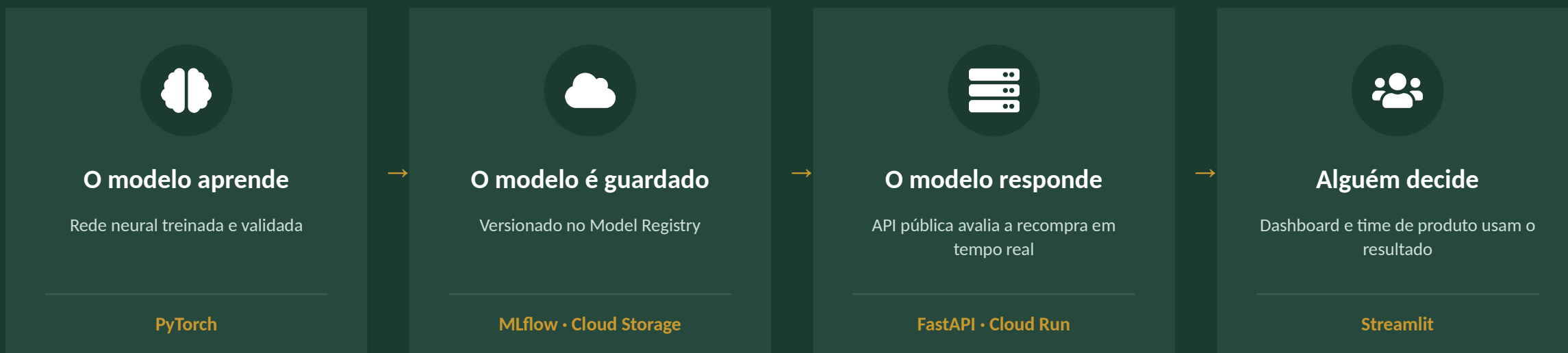
Treinei uma rede neural híbrida (embeddings + MLP, em PyTorch) e comparei com um baseline clássico (Regressão Logística, só com features tabulares). As duas ficaram próximas em AUC — a decisão real apareceu quando olhei recall.

Modelo	AUC-ROC	Recall	Precision	F1
Baseline (Regressão Logística)	0,8964	0,7698	0,8777	0,8202
Modelo Neural (Híbrido)	0,9045	0,9876	0,7879	0,8765

O modelo neural captura quase todos os casos reais de recompra (recall de 98,8%) — para recomendação, deixar de sugerir um produto relevante custa mais caro do que uma sugestão a mais.

Da ideia ao cliente: o caminho que os dados percorrem

O modelo treinado não serve para nada parado num notebook. Construimos o caminho completo para que ele chegasse a quem precisa decidir.



Quando um modelo melhor for treinado, ele entra nesse mesmo caminho — sem reconstruir nada que já funciona.

Arquitetura — Sistema de Recomendação Instacart

The diagram illustrates the Google Cloud Platform architecture for a recommendation system, organized into four main sections: DADOS & TREINAMENTO, Google Cloud Platform, USO / APLICAÇÕES, and CI/CD PIPELINE.

DADOS & TREINAMENTO

- 1. Dados**
Instacart Market Basket — 32,4M linhas (order_products__prior)
- 2. Treinamento**
PyTorch · embeddings(32) + MLP[128,64,32] · pipeline DVC (seed fixa)
- 3. Experimentos**
MLflow · tracking + Model Registry (Staging → Production)
- 4. Qualidade**
pytest (55 testes) · ruff (lint) · Pydantic (schemas)

Google Cloud Platform

SERVIÇOS GERENCIADOS (CLOUD RUN)

- recommender-api**
API de Inferência (FastAPI)
 - / · /health · /ready
 - /predict
 - /predict/batch
 - /metadata · /metrics
- recommender-dashboard**
Dashboard (Streamlit)
 - Consome a API
 - Predição única
 - Upload de lote (CSV)
 - Resultados priorizados

→ HTTP

CAMADA DE DADOS

- Cloud Storage**
Bucket de modelos por versão
- Container Registry**
Imagens Docker versionadas
- model_metrics.json**
Métricas do treino

OBSERVABILIDADE E MONITORAMENTO

- /metrics**
latência e throughput
- Logging**
JSON estruturado
- Alertas**
latência, erro 5xx
- Model Card**
métricas e vieses

USO / APLICAÇÕES

Dashboard (Streamlit)

- Predição única (formulário)
- Upload de lote (CSV)
- Download do resultado priorizado

Integrações via API

- Sistemas de e-commerce
- Workflows de recomendação
- Automação de marketing

Usuários

- Time de produto / e-commerce
- Analistas de dados

CI/CD PIPELINE

- 1. Code Commit**
- 2. Testes**
- 3. Build Imagem**
- 4. Cloud Build**
- 5. Deploy**
- 6. Versiona Modelo**

RESULT

O que essa jornada deixou pronto?

98,8%

de recall — quase nenhum caso real de recompra passa despercebido

+28%

mais recompra real capturada, frente a um modelo estatístico clássico

82% menor

custo de infraestrutura da API — deploys mais rápidos e baratos

no ar, agora

API e dashboard públicos, prontos pra integrar com qualquer e-commerce

A lição que mais importou: a métrica que parecia decidir tudo (AUC) quase não mudava entre os dois modelos — a diferença real apareceu quando parei de olhar acerto médio e passei a contar quantas oportunidades de venda um recall baixo deixaria escapar.



Tiago de Freitas Faustino

Tech Challenge 2 · FIAP MLE10 · Machine Learning Engineering

Obrigado! Vamos conversar sobre a arquitetura, os resultados ou os próximos passos.



github.com/tiagotff/FIAP_MLE10_TC2



linkedin.com/in/tiagofaustino91

PROJETO EM PRODUÇÃO

API

recommender-api-761613283146.us-central1.run.app

Dashboard

recommender-dashboard-761613283146.us-central1.run.app

Documentação (Swagger UI)

recommender-api-761613283146.us-central1.run.app/docs